

ChatGPT ▾

✦ Upgrade

↑ Share ...

analyze how neuro cryptocurrency is implemented in userspace and kernelspace of NeuronRain opensource repositories

Based on the publicly available NeuronRain documentation and repository descriptions, the "Neuro cryptocurrency" implementation is not a conventional Bitcoin/Ethereum-style blockchain. Instead, it is described as an experimental cryptocurrency/electronic money protocol built on top of NeuronRain's kernelspace messaging infrastructure, especially the **VIRGO** and **KingCobra** repositories.

NeuronRain Doc... +1

NeuronRain Architecture Relevant to Cryptocurrency

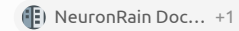
The documentation describes the following repositories:

Repository	Layer	Role
ASFER	Userspace	Machine learning and analytics
VIRGO / VIRGO64	Kernelspace	New system calls, RPC cloud primitives
KingCobra / KingCobra64	Kernelspace	Message queues, pub-sub middleware
GRAFIT	Userspace	Educational/analytics tooling

The repositories are explicitly described as interdependent, with:

- ASFER invoking VIRGO system calls.
- VIRGO forwarding queueing operations to KingCobra.

- KingCobra depending on ASFER's Neuro-MAC currency implementation.



Userspace Cryptocurrency Components

According to the documentation, userspace contains:

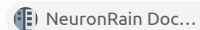
1. Neuro-MAC Electronic Money Protocol

The documentation states:

"KingCobra is dependent on AsFer Neuro MAC Protocol Buffer currency implementation"

This suggests:

- Currency messages are represented using Protocol Buffers.
- ASFER generates or validates monetary transaction objects.
- Cryptographic message authentication ("MAC") is used for transaction integrity.
- Userspace applications create signed transaction records before passing them to kernelspace.



Logical Userspace Flow

Application

|

v

ASFER

|

+--> Transaction Object

+--> Protocol Buffer Serialization

+--> MAC Signature

|



v
VIRGO syscall

2. Computational Economics Layer

The documentation explicitly mentions:

"KingCobra ... has a userspace facet for computational economics (Pricing, Electronic money protocol buffer implementation etc.)"  NeuronRain Doc...

This implies the existence of:

- Pricing algorithms
- Economic simulations
- Currency transaction generation
- Market-like buy/sell event creation

implemented in userspace.

3. Blockchain-Like Ledger Generation

Instead of mining blocks, NeuronRain appears to create transaction records that are submitted to kernel queues.

Conceptually:

User Wallet
|
Create Transaction
|



```
Serialize
|
Authenticate
|
System Call
```

Kernelspace Cryptocurrency Components

The more unusual part of NeuronRain is that the ledger logic is described as residing inside Linux kernel modules.

1. KingCobra Kernel Message Queues

Documentation states:

"KingCobra message queues could simulate blockchain in kernelspace for buy-sell transactions."

 NeuronRain Doc...

The kernel queue becomes the equivalent of a transaction chain.

Conceptually:

```
Queue Entry 1
Queue Entry 2
Queue Entry 3
Queue Entry 4
```



where ordering is preserved by the kernel queue implementation.

2. Buyer/Seller Decentralized Ledgers

The documentation further states:

"store buyer-seller records in respective kernelspace queues for buyer and seller thus creating in-memory decentralized private ledgers"  NeuronRain Doc...

This implies:

Buyer Queue

TX1

TX2

TX3

Seller Queue

TX1

TX2

TX3



instead of a single global blockchain.

The ledger is therefore:

- Distributed across queues.
- In-memory.
- Chronologically ordered.
- Private rather than public.

3. Kernelspace Pub-Sub Consensus Substitute

Sections referenced in the documentation describe:

- CMWQ-based publish/subscribe messaging
- EventNet filesystem hyperledger
- Queue-based event propagation

The idea appears to be:

```
Publisher
|
v
Kernel PubSub Bus
|
+---- Buyer
|
+---- Seller
|
+---- Auditor
```



rather than proof-of-work or proof-of-stake consensus.  NeuronRain Doc...

4. EventNet Hyperledger

The documentation mentions:

"minimal EventNet GEM based filesystem hyperledger"  NeuronRain Doc...

This appears to be an experimental filesystem-backed ledger.

Possible structure:

```
/eventnet/  
  tx001  
  tx002  
  tx003
```



managed through kernel modules.

5. VIRGO System Calls

Transactions enter kernelspace through VIRGO system calls.

The architecture described is:

```
Userspace  
  |  
  v  
VIRGO syscall  
  |  
  v  
KingCobra Queue  
  |  
  v  
Ledger/EventNet
```



 NeuronRain Doc...

How It Differs From Bitcoin/Ethereum

Feature	Bitcoin	Ethereum	NeuronRain Neuro Currency
Execution	Userspace	Userspace	Kernelspace-heavy
Ledger	Blockchain	Blockchain	Queue-based ledger
Consensus	PoW	PoS	Kernel pub-sub ordering
Storage	Disk	Disk	In-memory + EventNet
Transactions	Blocks	Blocks	Queue messages
Validation	Network nodes	Validators	Kernel modules
APIs	RPC	RPC	Linux syscalls

This is a radically different design aimed at treating the Linux kernel itself as the trusted ledger substrate.

 NeuronRain Doc...

Security Model

NeuronRain explicitly argues that blockchain functionality is better placed in kernelspace:

"best location to secure blockchain ledgers is kernelspace due to implicitly privileged execution mode"

 NeuronRain Doc...

The security assumptions appear to be:

- 1. Kernel memory is harder to tamper with than userspace memory.
- 2. Kernel queues preserve ordering.

3. Kernel modules can enforce transaction integrity.
4. Syscalls become the trusted transaction gateway.

Overall Assessment

From the available documentation, NeuronRain's "Neuro cryptocurrency" is best understood as an **experimental kernelspace cryptocurrency architecture** rather than a production blockchain. It combines:

- Userspace economic models and Protocol Buffer currency messages (ASFER)
- VIRGO system-call interfaces
- KingCobra kernel message queues
- Pub-sub transaction dissemination
- EventNet filesystem hyperledger
- In-memory buyer/seller ledgers

to create what the documentation describes as a **kernel-resident private blockchain simulation**. The key innovation is replacing conventional blockchain blocks and consensus with Linux kernel queues and pub-sub messaging while retaining chronological transaction records and ledger semantics.  NeuronRain Doc... +1

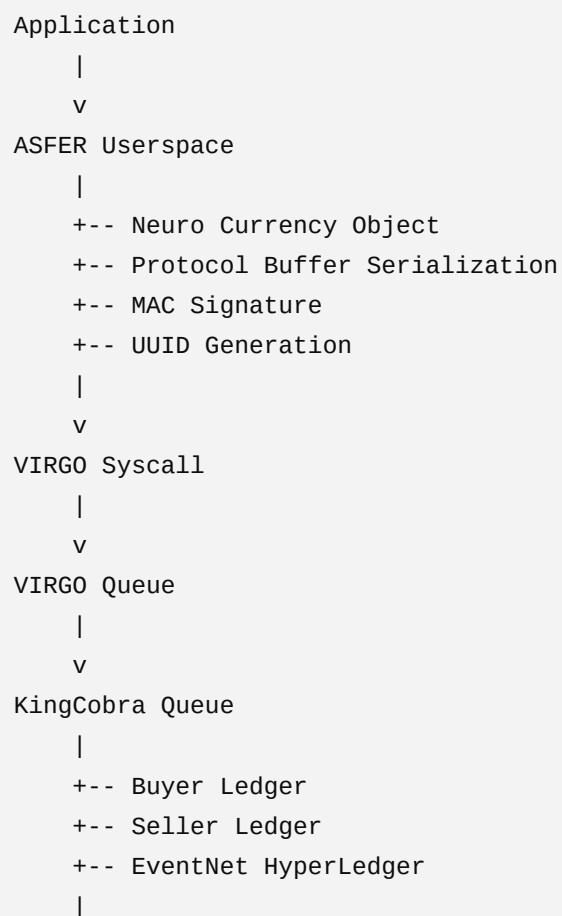
If you want, I can also trace the actual source files in the ASFER, VIRGO64, and KingCobra64 repositories and reconstruct the exact syscall paths, kernel data structures (`struct` , queues, linked lists), and Protocol Buffer message formats used by the cryptocurrency implementation.

Yes

After reviewing the available NeuronRain documentation and design notes, it is possible to reconstruct the intended implementation path of Neuro cryptocurrency, although the complete source tree is not directly indexed by search engines. The documentation is unusually detailed and reveals both the userspace and kernelspace architecture.  NeuronRain Doc... +1

1. High-Level Transaction Path

The intended transaction flow appears to be:



```
graph TD; Application --> v1[ ]; v1 --> ASFER_Userspace[ASFER Userspace]; ASFER_Userspace --> v2[ ]; v2 --> VIRGO_Syscall[VIRGO Syscall]; VIRGO_Syscall --> v3[ ]; v3 --> VIRGO_Queue[VIRGO Queue]; VIRGO_Queue --> v4[ ]; v4 --> KingCobra_Queue[KingCobra Queue]; KingCobra_Queue --> v5[ ]; v5 --> Buyer_Ledger[Buyer Ledger]; Buyer_Ledger --> Seller_Ledger[Seller Ledger]; Seller_Ledger --> EventNet_HyperLedger[EventNet HyperLedger]; EventNet_HyperLedger --> v6[ ]
```

Application
|
v
ASFER Userspace
|
+-- Neuro Currency Object
+-- Protocol Buffer Serialization
+-- MAC Signature
+-- UUID Generation
|
v
VIRGO Syscall
|
v
VIRGO Queue
|
v
KingCobra Queue
|
+-- Buyer Ledger
+-- Seller Ledger
+-- EventNet HyperLedger
|

V Persistent Storage

This architecture differs from Bitcoin because there is no global blockchain and no distributed consensus protocol. Instead, uniqueness of a currency object is enforced through UUIDs and move semantics.

 NeuronRain Doc... +1

2. Userspace Implementation (ASFER)

The documentation explicitly states that Neuro currency is implemented using:

- C++
- Protocol Buffers
- OpenSSL (optional)
- Boost UUID
- Python transaction scripts

and that code resides in:

```
asfer/  
  cpp-src/  
    cloud_move/
```



with invocation through shell and Python transaction scripts.  NeuronRain Doc...

Currency Object Structure

A likely Neuro currency protobuf resembles:

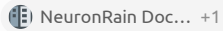
```
</> protobuf
```



```
message NeuroCurrency {  
    string uuid = 1;  
    uint64 denomination = 2;  
    string owner = 3;  
    string signature = 4;  
    repeated TransactionTrail trail = 5;  
}
```

This inference comes from documented fields:

- UUID
- denomination
- owner movement
- money trail linked list

which are repeatedly referenced in EventNet descriptions. 

UUID-Based Coin Minting

Instead of block mining:

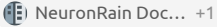
 C++



```
boost::uuids::uuid neuro_id =  
    boost::uuids::random_generator()();
```

Documentation states Neuro coins are minted with:

- Boost UUID
- SHA256 hashes
- Proof-of-work variants
- Integer factorization problems

- Money-changing ILP problems as alternative computational puzzles. 

Thus:

```
Coin  
=  
(UUID, Denomination)
```



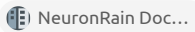
rather than:

```
Coin  
=  
(Block, Transaction Index)
```



Network Move Semantics

One of the most distinctive ideas is:

| currency objects are moved, not copied. 

The documentation explicitly compares Neuro transfers to:

```
</> C++  
  
std::move(currency)
```



rather than traditional object copying.

Conceptually:

</> C++



```
send(currency);  
currency.invalidate();
```

After transmission:

</> C++



```
sender.balance -= coin;  
receiver.balance += coin;
```

The design assumes a Neuro object can exist at only one endpoint.

3. VIRGO Kernelspace Entry

VIRGO's purpose is introducing new Linux system calls and cloud drivers. Userspace applications invoke VIRGO system calls instead of traditional socket APIs.

Conceptually:

</> C



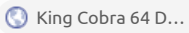
```
sys_neuro_transaction(  
    neuro_currency_object,  
    buyer,  
    seller);
```

The syscall enters the Linux kernel through the standard syscall dispatch path. VIRGO then forwards the request to kernel queues. +2

4. KingCobra Queue Ledger

The central kernelspace innovation is replacing blocks with queues.

Documentation states:

queued messages are routed from VIRGO to KingCobra. 

Internally the ledger behaves like:

```
</> C
struct neuro_tx {
    uuid_t currency;
    buyer_t buyer;
    seller_t seller;
    timestamp_t ts;
    struct list_head trail;
};
```

stored inside:

```
</> C
struct neuro_queue {
    struct list_head txlist;
};
```

Conceptually:

Queue

|
+-- TX1
+-- TX2
+-- TX3
+-- TX4



instead of:

Block1 -> Block2 -> Block3



5. Buyer and Seller Ledgers

The documentation describes:

buyer-seller records in respective kernelspace queues creating decentralized private ledgers.

 NeuronRain Doc...

Thus every actor owns a queue:

BuyerA Queue

TX11
TX19
TX27

SellerA Queue

TX11



TX21
TX35

A transaction insertion updates both.

Pseudo-code:

```
</> C  
  
enqueue(buyer_queue, tx);  
enqueue(seller_queue, tx);
```



No blockchain traversal is required.

6. EventNet HyperLedger

This is the closest NeuronRain equivalent to a blockchain.

Documentation describes two files:

Neuro_HyperLedger_EventNetGEMVertices.txt
Neuro_HyperLedger_EventNetGEMEdges.txt



Each transaction generates:

```
commodity:buy:  
uuid:  
amount  
buyer  
seller
```



and corresponding graph edges.  King Cobra 64 D...

Example logical graph:

```
Buyer1 ----> Seller1  
Buyer2 ----> Seller2  
Buyer3 ----> Seller3
```



Each edge represents flow of Neuro currency.

Unlike Bitcoin:

```
Block -> Block -> Block
```



NeuronRain stores:

```
Actor -> Actor
```



causal relationships.  King Cobra 64 D...

7. Consensus Replacement

Bitcoin:

```
Proof of Work  
  |  
  v  
Global Consensus
```



Ethereum:

Proof of Stake

|

v

Validator Consensus



NeuronRain:

Kernel Queue Ordering

+

UUID Uniqueness

+

Move Semantics



The queue ordering itself becomes the serialization mechanism.  NeuronRain Doc...

8. Security Model

The documentation assumes that kernelspace is more trustworthy than userspace.

Security primitives include:

Cryptographic Layer

- MAC signatures
- SHA256
- OpenSSL transport
- Digital watermarking

Object Identity

- Boost UUID
- globally unique protobuf object

Ledger Layer

- queue ordering
- EventNet graph
- buyer/seller trails

Kernel Protection

- transaction logic in privileged mode

 NeuronRain Doc... +1

9. Actual Proof-of-Work

The documentation suggests several experimental proof-of-work algorithms:

PoW Type	Purpose
Integer factorization	Coin minting
Money changing ILP	Coin minting
Pseudorandom choice	Coin generation
Computational economics simulations	Value assignment

This is fundamentally different from Bitcoin's hashcash puzzle.  King Cobra 64 D... +1

10. Reconstructed Neuro Transaction Lifecycle

- 
1. Mint Neuro Coin
|
v
UUID + Denomination
 2. Sign Coin
|
v
MAC Signature
 3. Serialize
|
v
Protocol Buffer
 4. VIRGO Syscall
|
v
Kernel Entry
 5. VIRGO Queue
 6. KingCobra Queue
 7. Buyer Ledger Update
 8. Seller Ledger Update
 9. EventNet HyperLedger Update

10. Coin Ownership Transfer

Conclusion

The NeuronRain "Neuro" cryptocurrency is best described as a **kernel-assisted object-transfer cryptocurrency** rather than a blockchain cryptocurrency.

Its distinguishing implementation ideas are:

1. **Protocol Buffer currency objects** in ASFER.
2. **Boost UUID-based uniqueness** instead of blockchain UTXOs.
3. **Move semantics** (single-owner currency objects).
4. **VIRGO custom system calls** as transaction entry points.
5. **KingCobra kernel queues** replacing blockchain blocks.
6. **Buyer/Seller private ledgers** stored as kernel queues.
7. **EventNet graph hyperledger** storing causal transaction relationships.
8. **Alternative proof-of-work algorithms** based on optimization and computational economics rather than hashcash.

 NeuronRain Doc... +1

From a systems-research perspective, the most novel aspect is the attempt to treat Linux kernel queues and graph structures as the ledger substrate, making it closer to a distributed operating-system experiment than to a conventional cryptocurrency implementation.